تصميم برامج الحاسب الالى
مراجعة

د/الشيماء عبدالرؤوف

دورة حياة تطوير البرمجيات :

- هو **مجموعة من المراحل المنظمة لتطوير البرمجيات** من الفكرة حتى التسليم والصيانة
- **مراحل دورة حياة تطوير البرمجيات:**

جمع وتحليل المتطلبات: فهم ما يحتاجه المستخدم وتحديد الوظائف والخصائص المطلوبة.

التصميم: تصميم بنية البرنامج وتحديد كيفية عمل النظام داخليًا وخارجيًا.

البرمجة: كتابة الكود باستخدام لغات البرمجة المناسبة وتحويل التصميم إلى برنامج فعلي.

الاختبار :التأكد من أن البرنامج يعمل بدون أخطاء واختبار كل وظيفة للتأكد من تحقيق المتطلبات

.التنفيذ والنشر: تثبيت البرنامج على بيئة المستخدم.التأكد من أنه يعمل بشكل صحيح في الواقع.

الصيانة والدعم: إصلاح الأخطاء بعد النشر

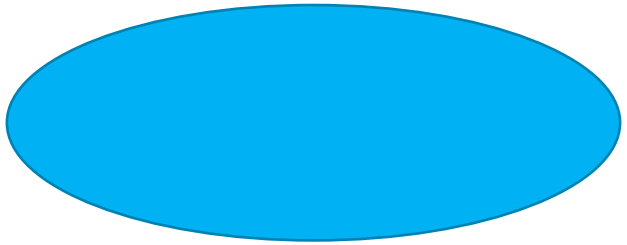
طريقة تمثيل الخوارزميات

- الشفرة المزيفه Pseudo code
- خرائط سير العمل Flow chart
- استخدام اللغات الطبيعه

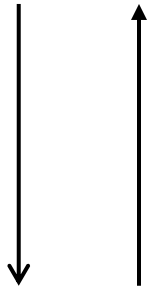
مقارنة بين طريقة تمثيل الخوارزميات

العنصر	اللغات الطبيعية	الخوارزم والكود الزائف	المخططات الانسيابية
التعريف	التعبير عن خطوات الحل باستخدام اللغة العادية مثل العربية أو الإنجليزية	كتابة خطوات الحل بشكل منظم يشبه البرمجة بدون التقيد بلغة معينة	تمثيل خطوات الحل باستخدام رموز وأشكال
طريقة الكتابة	جمل وعبارات وصفية	خطوات مرتبة	رموز متصلة بأسهم
الدقة	قليلة نسبيًا	عالية	عالية جدًا
سهولة الفهم	سهلة للبشر	متوسطة	سهلة جدًا بصريًا
الطول	قد تكون طويلة	مختصرة نسبيًا	مختصرة
الاعتماد على اللغة	تعتمد على أسلوب الشخص	لا تعتمد على لغة معينة	لا تعتمد على اللغة
الاستخدام	شرح مبدئي بسيط	إعداد الحل قبل البرمجة	توضيح سير العمليات بصريًا
المميزات	سهلة وبسيطة	منظمة وقريبة من البرمجة	واضحة وسهلة الفهم
العيوب	- طويلة - غير دقيقة - تختلف من شخص لآخر	تحتاج تفكير وتنظيم	تحتاج معرفة الرموز

رموز خرائط التدفق



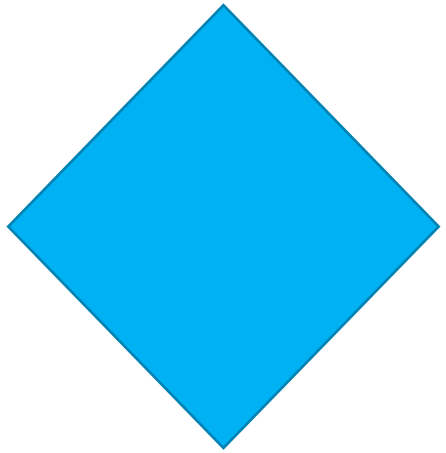
تستخدم لتحديد البداية والنهاية وتكون احد قيميتين البداية و النهاية



تحديد اتجاه سير البرنامج

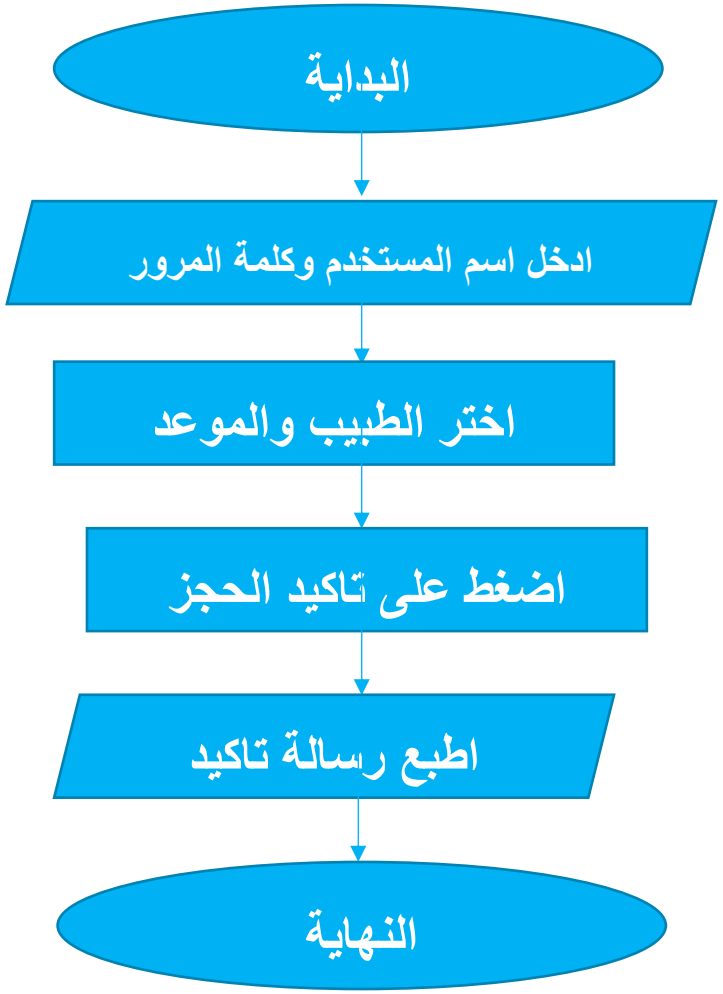
عملية او معالجة

الادخال الاخراج البيانات



للتعبير عن العمليات المنطقيه

مثال :حجز موعد طبيب اون لاين



المدخلات:

اسم المستخدم ، كلمة المرور اختيار الطبيب ، تحديد الموعد

المخرجات:

تأكيد حجز الموعد

الخطوات:

1. البداية
2. أدخل اسم المستخدم وكلمة المرور
3. اختر الطبيب وحدد الموعد المناسب
4. اضغط على تأكيد الحجز
5. اطبع رسالة تأكيد الحجز
6. النهاية

الفرق بين المترجم compiler والمفسر interpreter

لنبسط الفرق بينهما
بالجدول التالي

Compiler	Interpreter
يقوم بتحليل البرنامج كامل ثم ترجمته كاملاً	يقوم بتحليل البرنامج سطرا سطر من الكود المصدري
التنفيذ سريع جداً	أبطئ بالتنفيذ
يكتشف الأخطاء بعد تحليل البرنامج كامل	يسهل تتبع الأخطاء لأنه يتوقف عند الخطأ مباشرة

أمثله للغات تستخدم

Compiler	Interpreter
C	Python
C++	PHP
C#	Perl
	Ruby

تسمية المتغيرات

القاعدة	التوضيح
تعريف المتغير	مكان في الذاكرة لتخزين قيمة
طول الاسم	من 1 إلى 31 حرف/رقم أو underscore
بداية الاسم	يجب أن يبدأ بحرف أو underscore
عدم وجود مسافات	لا يُسمح بمسافات داخل الاسم
الرموز المسموح بها	فقط (_) underscore
استخدام الأرقام	مسموح لكن ليس في البداية

النوع	إستخدامه
Int رقم صحيح	يستخدم لتخزين أعداد صحيحة. مثال x = 3
Float رقم عشرى	يستخدم لتخزين أعداد تحتوي على فاصلة عشرية. مثال x = 1.5
Complex اعداد مركبة	يستخدم لتخزين أعداد مركبة (Complex Number) و التي غالباً ما يحتاجها المهندسون عند إجراء عمليات حسابية معقدة. و هنا يجب وضع الحرف J أو j بعد العدد حتى يعلم مفسّر بايثون أننا نقصد عدد مركب و ليس عدد عادي. مثال x = 4J
السلاسل النصيه string	النصية (string) هي عبارة عن تسلسل من محرف واحد أو أكثر (محارف وأعداد ورموز)، ويمكن أن تكون ثابتة أو متغيرة X="hello"
القيم المنطقيه bool	عند إسناد القيمة True أو القيمة False إلى المتغير فإنه يصبح من النوع bool. x=true

حدد نوع المتغير فيما ياتى وهل اسم المتغير صحيح ام خاطىء:

اسم المتغير	القيمة	النوع	صحيح أم خطأ	السبب
age	30	int	صح ✔	يبدأ بحرف، لا مسافات أو رموز
salary	350.25	float	صح ✔	يبدأ بحرف، لا رموز غير مسموح بها
name	"Ahmed"	string	صح ✔	يبدأ بحرف، لا مسافات، قيمة نصية صحيحة
is_student	False	bool	صح ✔	يبدأ بحرف، underscoreمسموح، نوع قيمة صحيح
counter	10	int	صح ✔	صحيح، يبدأ بحرف
_temp	23.5	float	صح ✔	يبدأ بـ (_) underscoreمسموح
first_name	"Ali"	string	صح ✔	يحتوي (_) underscoreمسموح
is_active	True	bool	صح ✔	يبدأ بحرف، underscoreمسموح
2ndPlace	1	int	خطأ ✘	يبدأ برقم → غير مسموح
full name	"Sara"	string	خطأ ✘	يحتوي مسافة → غير مسموح
total$	500	int	خطأ ✘	يحتوي رمز غير مسموح ($)
marks#	80	int	خطأ ✘	يحتوي رمز غير مسموح (#)
veryLongVariableNameThatExceeds31Chars	5	int	خطأ ✘	الاسم أطول من 31 حرف → غير مسموح
first-name	"Ahmed"	string	خطأ ✘	يحتوي رمز (-) غير مسموح

ترتيب العمليات الحسابية من الأعلى للأدنى

- في لغة Python يتم تنفيذ العمليات الحسابية بالترتيب التالي:
- 1 **الأقواس ( )**
  2 **الأسس ****
  3 **الضرب والقسمة * / // %**
  4 **الجمع والطرح + -**

عند تساوي العمليات في نفس المستوى يتم التنفيذ من **اليسار إلى اليمين**

ماناتج جملة الطباعة الاتيه:

- print(7 + 5 * 3)

- الحل خطوة بخطوة:
- 5 × 3 = 15
- 7 + 15 = 22
- الناتج = **22**

print((8 + 2) * 5 - 12 / 4)

- لحل خطوة بخطوة:
- القوس أولًا: 8 + 2 = 10
- الضرب: 10 × 5 = 50
- القسمة: 12 ÷ 4 = 3
- الطرح: 50 - 3 = 47
- الناتج = **47**

الدالة input

- input()هي دالة تستخدم **لأخذ البيانات من المستخدم أثناء تشغيل البرنامج.**
- تُرجع **نصًا** دائمًا، حتى لو كتب المستخدم رقمًا.
- لتحويلها لأرقام نستخدم: int او float

```
age = int(input(" :اكتب عمرك"))
height = float(input(" :اكتب طولك بالمتر"))
```

جمل الشرط

if :

#إذا كان الشرط صحيحاً نفذ هذا الكود

elif :

#إذا كان الشرط صحيحاً نفذ هذا الكود

else:

#إذا لم يتحقق أي شرط نفذ هذا الكود

اى من التعابير الاتية صحيح وايها خطا

- if y = 7: ✗
- if x == 10: ✓
- z == 3 ✗
- x = 10 ✓
- c = 5 + 2 ✓
- i=i+1 ✓
- i+=1 ✓

مثال: برنامج لتحديد سعر التذكرة حسب العمر والفئة الطلابية
يطلب من المستخدم إدخال عمره.
إذا كان العمر أقل من 12 → التذكرة مجانية.
إذا كان العمر بين 12 و 18 → التذكرة بنصف السعر.
إذا كان العمر أكبر من 18 → التذكرة كاملة.

```
age = int(input(":أدخل عمرك”)
if age < 12:
    print("التذكرة مجانية”)
elif age <= 18:
    print("التذكرة بنصف السعر”)
else:
    print("التذكرة كاملة”)
```

العوامل التى تستخدم فى وضع اكتر من شرط منطقى

رمزه	مثال	شرح الكود
and	a and b	هل قيمة a و b تساويان True ؟ هنا يجب أن يتم تحقيق الشرطين ليرجع True
or	a or b	هل قيمة a أو b أو كلاهما تساويان True ؟ هنا يكفي أن يتم تحقيق شرط واحد من الشرطين ليرجع True
not	not a	هل قيمة a لا تساوي True ؟ إذا كان الجواب نعم فإنها ترجع True

ماناتج البرنامج التالى

```
age = 20
grade = 88
courses = 4
if 18 <= age <= 25 and grade >= 85 and courses >= 3:
    print("يحصل على منحة كاملة")
elif 18 <= age <= 25 and (grade >= 70 or courses >= 2):
    print("يحصل على منحة جزئية")
else:
    print("لا يحصل على المنحة")
```

يحصل على منحة كاملة

ماناتج البرنامج التالى

```
grade = 88
projects = 4
if grade >= 90 or projects >= 3:
    print("الطالب مؤهل للمشاركة")
else:
    print("الطالب غير مؤهل للمشاركة")
```

الطالب مؤهل للمشاركه

الدالة range

- في لغة **Python** تستخدم الدالة **range()** لإنشاء **تسلسل من الأرقام**، وغالبًا تُستخدم مع حلقة **for** لتكرار الأوامر.
- الشكل العام:

- **range(مقدار الزيادة بين الأرقام, الرقم الذي يتوقف عنده, الرقم الذى يبدا منه العد)**

- range() لا يشمل الرقم الأخير.

السؤال: أي من العبارات التالية صحيحة أم خاطئة؟

- range(2, 6) → 5 ,4 ,3 ,2 ✅
- range(4) → 4 ,3 ,2 ,1 ,0 ❌
- range(3, 10, 3) → 9 ,6 ,3 ✅
- range(10, 2, -2) → 2 ,4 ,6 ,8 ,10 ✅
- range(1, 7, 2) → 6 ,5 ,4 ,3 ,2 ,1 ❌

الفرق بين for , while

الخاصية	for	while
الاستخدام الأساسي	تُستخدم لتكرار مجموعة محددة من العناصر أو عدد محدد من المرات	تُستخدم لتكرار الكود طالما أن شرط معين صحيح
متى نعرف عدد التكرارات مسبقًا؟	نعم، غالبًا يكون معروفًا (مثال: حلقة عبر قائمة أو range)	لا، قد يعتمد على شرط يتغير أثناء التنفيذ
مثال على التكرار	for i in range(5): print(i)	i = 0 while i < 5: print(i) i += 1
التحكم في الحلقة	يمكن استخدام break و continue	يمكن استخدام break و continue
الاستخدام الشائع	تكرار على عناصر قائمة، مجموعة، أو أرقام محددة	تكرار حتى يتحقق شرط معين، أو انتظار حدث معين

الفرق بين break , continue

الخاصية	break	continue
الوظيفة الأساسية	إيقاف الحلقة بالكامل والخروج منها فورًا	تخطي التكرار الحالي من الحلقة والانتقال للتكرار التالي
تأثيرها على الحلقة	تنهي الحلقة نهائيًا مهما تبقى من عناصر أو تكرارات	تستمر الحلقة في التكرارات القادمة بعد تخطي التكرار الحالي
الاستخدام الشائع	عندما يتحقق شرط معين لا داعي لاستكمال باقي التكرار	عندما يتحقق شرط معين ونريد تخطي هذا التكرار فقط

مثال	for i in range(5):
	if i == 3:
	break
	print(i)
	الناتج: 0 1 2
		for i in range(5):
		if i == 3:
		continue
		print(i)
		الناتج: 0 1 2 4

الفرق بين النصوص والقوائم والصفوف

الخاصية	List القوائم	Tuple الصفوف	String النصوص
التعريف	مجموعة مرتبة من العناصر قابلة للتغيير	مجموعة مرتبة من العناصر غير قابلة للتغيير	سلسلة من الأحرف (نص)
قابلة للتغيير	نعم يمكن تعديل، إضافة أو حذف العناصر	لا العناصر ثابتة بعد التعريف	لا النصوص ثابتة بعد التعريف
الأقواس المستخدمة	[ ]	( )	' ' أو " "
أنواع البيانات	يمكن أن تحتوي على أي نوع بيانات مختلط	يمكن أن تحتوي على أي نوع بيانات مختلط	تحتوي على أحرف فقط
أمثلة	[1, 2, 3]	(1, 2, 3)	"Hello"
الوصول للعناصر	عن طريق الفهرس مثل mylist[0]	عن طريق الفهرس مثل mytuple[0]	عن طريق الفهرس مثل mystring[0]
الدعم للعمليات	يمكن التغيير، الإضافة append(), extend(), pop()	لا يمكن التغيير، يمكن فقط القراءة	يمكن استخدام عمليات نصية مثل len(), lower(), split()
الاستخدام الشائع	قوائم قابلة للتعديل لتخزين بيانات متعددة	تخزين بيانات ثابتة لا تحتاج للتغيير	تمثيل نصوص ومعالجتها

ماناتج البرنامج التالى

```
temperatures = [18, 22, 25, 28, 35, 30, 20]
for temp in temperatures:
if temp < 20:
        continue
if temp >= 35:
        break
print(" مسجلة درجة الحرارة“,temp)
```

مسجلة 22Cدرجة الحرارة
مسجلة 25Cدرجة الحرارة
مسجلة 28Cدرجة الحرارة

حاول ان تحل

- my_list = [5, 15, 25, 35, 45, 55]

my_list[2]	25
my_list[2:5]	[25, 35, 45]
my_list[:4]	[5, 15, 25, 35]
my_list[3:]	[35, 45, 55]
my_list[:]	[5, 15, 25, 35, 45, 55]

ضع صح او خطا

my_tuple = (5, 8, 12)

my_list = [5, 8, 12]

- my_tuple[1] = 20 ❌
- my_tuple.append(15) ❌
- my_tuple.pop() ❌
- my_tuple.pop[0] ❌
- my_list[1] = 20 ✅
- my_list.append(15) ✅
- my_list.pop() ✅
- my_list.pop(0) ✅

معرفة عدد أحرف النص

- لمعرفة عدد أحرف النص نقوم باستدعاء الدالة **len()** و نمرر لها النص فقط.

- print('Length of s ', len(s))

- الناتج 5

مثال : اكتب برنامج يتحقق من طول اسم المدينة إذا كان طول الاسم أكثر من 10 أحرف، اطبع: "اسم المدينة طويل جدًا".إذا كان طول الاسم أقل من 5 أحرف، اطبع: "اسم المدينة قصير جدًا".إذا كان طول الاسم بين 5 و10 أحرف، اطبع: "اسم المدينة مناسب".

```
city_name = "الإسكندرية"
if len(city_name) > 10:
    print("اسم المدينة طويل جدًا")
elif len(city_name) < 5:
    print("اسم المدينة قصير جدًا")
else:
    print("اسم المدينة مناسب")
```